

SFA – Operations and Infrastructure

Overview

SFA is a production system running on a volunteer-operated infrastructure stack. The production pipeline runs daily on a home server (waldhomeserver), publishes to a managed WordPress hosting service (uPress), and delivers price data to a public WordPress page. This document covers everything from the physical hardware to the public delivery layer.

Infrastructure Stack

waldhomeserver – Production Home Server

Attribute	Value
Hardware	Intel i5 (4-core), 8 GB RAM, 256 GB SSD + 1 TB HDD
OS	Ubuntu 24.04 LTS
Always-on	Yes – network-connected 24/7
Access	SSH via Tailscale VPN (<code>waldhomeserver.home-nimrod.ts.net</code>) or direct IP (<code>100.125.98.56</code>)
SSH user	<code>nimrodw</code>
Networking	Tailscale mesh VPN (<code>10.100.102.2</code> on the VPN subnet)
Services	sshd, tailscaled, Docker (PostgreSQL), systemd unit <code>sfa-admin</code> , cron scheduler

The waldhomeserver is not a cloud server. It is a physically located home server running continuously. This is a deliberate cost decision: for a volunteer community project, server costs matter. A home server eliminates hosting fees while providing always-on availability.

Resilience tradeoff: Home servers are subject to power outages, ISP disruptions, and hardware failures in ways that cloud servers are not. SFA’s publish architecture is designed to tolerate this: the WordPress site reads from static versioned files, and `manifest_last_good.json` serves as a fallback when the latest publish is unavailable.

Development Environment (Mac Workstation)

Attribute	Value
OS	macOS
Database	PostgreSQL via Docker (port 5433 – distinct from production port to prevent conflicts)
Flask admin	Port 5001
Scheduler policy	Manual only – no automatic FTPS upload on dev machines
Code access	Full – all CLI commands available, all pipeline stages executable

The separation between dev and production is enforced by policy: the FTPS upload command is available in development but disabled from running automatically. This prevents a developer running a test pipeline from accidentally overwriting production artifacts.

uPress.co.il – Public WordPress Hosting

Attribute	Value
Provider	uPress.co.il (Israeli WordPress managed hosting)
Server	S887
Upload path	<code>wp-content/uploads/market/</code>
WordPress version	Current
Theme	Flatsome (parent + child theme)
Cache	ezCache (WordPress cache) – purged after each successful upload
TLS	Required for FTPS – custom TLS session reuse implementation

The FTPS challenge: Standard Python FTPS clients fail against uPress's server configuration. The server requires TLS session reuse between the control connection and the data connection – a security requirement that most Python FTPS libraries do not implement

correctly. SFA uses a custom `ReusedSessionFTP_TLS` subclass of Python's standard library `ftplib.FTP_TLS` that overrides the connection initialization to correctly reuse the TLS session. Without this fix, every upload attempt returns `425 Can't open data connection`. This is not a general fix – it is specifically tuned for uPress's server behavior.

The Daily Pipeline in Production

The production pipeline on waldhomeserver runs as a cron-triggered daily cycle:

```
[cron trigger, ~06:00]
  ↓
run_ingestion          (Collectors + Parsers – fetches all active sources)
  ↓
run_normalize          (Normalizer – processes all new raw_extracted_items)
  ↓
run_aggregate          (Aggregator + QA Engine – computes daily statistics)
  ↓
run_publisher          (Build artifacts – JSON + HTML files)
  ↓
run_publisher --upload  (FTPS upload to uPress)
  ↓
[ezCache purge + manifest verification]
```

Each stage is **self-gating** – if a stage finds nothing to process, it exits cleanly without running. If ingestion produces no new raw assets (because sources haven't changed since the last run), normalization is skipped. This prevents the pipeline from processing stale data.

The Publish Artifacts

Every publish run creates five files:

Artifact	Description
<code>public_report-{ts}.json</code>	Machine-readable price index with full metadata, all product data, data quality snapshot
<code>public_report-{ts}.html</code>	Standalone HTML viewer (for direct access, not embedded in WordPress)
<code>public_report_body-{ts}.html</code>	The WordPress embed fragment – scoped CSS + price table HTML
<code>manifest.json</code>	Current pointer – links to latest artifacts, includes staleness and schema version
<code>manifest_last_good.json</code>	Copy of the last successful manifest – fallback for resilience

Plus fixed-name aliases:

- `public_report.json` → `public_report-{ts}.json`
- `public_report_body.html` → `public_report_body-{ts}.html`

The fixed-name aliases allow the WordPress embed to always load

`public_report_body.html` without knowing the timestamp, while the timestamped files preserve the full version history.

Manifest Schema

The `manifest.json` drives the WordPress frontend's knowledge of what's available:

```
{
  "schema_version": "2",
  "artifact_version": "20260423-060000",
  "staleness_days": 0,
  "staleness_level": "ok",
  "generated_at": "2026-04-23T06:03:12Z",
  "artifacts": {
    "public_report": "public_report-20260423-060000.json",
    "public_report_body": "public_report_body-20260423-060000.html"
  },
  "fixed_names": {
    "public_report.json": "public_report-20260423-060000.json",
    "public_report_body.html": "public_report_body-20260423-060000.html"
  },
  "data_quality": {
    "total_products": 67,
    "products_with_data": 62,
    "total_observations": 174,
    "active_sources": 7,
    "resolution_rate": 1.0
  },
  "upload_base": "https://www.nimrod.bio/wp-content/uploads/market"
}
```

The `data_quality` section gives any consumer of the manifest (including the WordPress frontend) visibility into the pipeline health at the time of publication.

WordPress Integration

The public price index is embedded in a WordPress page at `nimrod.bio/smallfarmsagent` using a custom WordPress plugin with a shortcode: `[sfagent_market_report]`.

How it works:

1. The shortcode renders a container `<div>` on the WordPress page
2. JavaScript in the page fetches `manifest.json` from the uploads directory
3. From the manifest, it retrieves the `public_report_body.html` artifact path
4. The HTML fragment is injected into the container

The `public_report_body.html` is a **self-contained** HTML fragment – it includes its own scoped `<style>` block using the `.sfagent-*` CSS prefix namespace. This prevents conflicts with the WordPress theme CSS.

Staleness handling: The WordPress frontend reads `staleness_level` from the manifest and displays an appropriate notice if the data is `warning` (3+ days) or `stale` (8+ days).

Fallback: If the manifest fetch fails, the frontend falls back to `manifest_last_good.json` before showing an error.

CSS Architecture

Three-layer CSS system for the WordPress integration:

Layer 1 – Flatsome Theme: The WordPress parent and child theme provides typography, layout, and button styles. These are not modified by SFA.

Layer 2 – `sfagent-base.css` : A shared CSS file hosted on the WordPress server, providing:

- CSS custom properties (tokens): `--green-dark`, `--green-light`, `--sand`, `--text-primary`, `--border`
- Base `.sfagent-*` component styles
- RTL layout foundation

Layer 3 – Inline `<style>` in `public_report_body.html` : Page-specific styles scoped to `.sfagent-*` prefix. Generated fresh with each publish run. Because it's in the published artifact rather than a separate file, it versions with the report.

Responsive breakpoint: 640px

- Desktop/tablet ($\geq 640\text{px}$): Full product table with metrics in columns – average price, median, range, stddev, observation count, source count
 - Mobile ($< 640\text{px}$): Stacked product cards – each product gets a card with the full metrics below the product name. Standard deviation column is hidden on mobile.
-

Admin UI Operations

The Flask admin UI (port 5001, Hebrew RTL) is the day-to-day operational interface. Common operator workflows:

Adding a new source:

1. Create a row in `data_sources` via the admin Sources page
2. Define the fetch profile (collector type, parser type, URL, headers, schedule)

3. Run a manual ingestion (`run_ingestion --source-code [new_source]`)
4. Review new `raw_extracted_items` in the Observations tab
5. Process unresolvables: add aliases or scope-skip rules as needed
6. Run normalize, aggregate, publish to verify the source integrates correctly
7. Mark source as active for automated scheduling

Handling an unresolvable item:

1. Observe the unresolvable item in the admin Observations page (filter by `status = unresolvable`)
2. Identify the product and source context
3. Decision: is this an in-scope product?
 - Yes → add alias in the Aliases page, then re-run normalize
 - No → add scope-skip rule in the Scope Skip Rules page, then re-run normalize
4. Verify the item transitions from `unresolvable` to `normalized` or `ignored`

Flagging a suspicious observation:

1. Find the observation in the Observations page
2. Apply `hide` flag – the observation is excluded from future aggregates without deletion
3. Or apply `review` flag – observation is marked for review but still included
4. Re-run aggregate to update daily statistics

Cross-Host Communication

The waldhomeserver and Mac workstation communicate via a file-based protocol over SSH/SCP:

- Mac outbox: `~/Documents/_agent_comm/outbox/`
- Server inbox: `~/agent_comm/inbox/`
- Server outbox: `~/agent_comm/outbox/`
- Mac inbox: `~/Documents/_agent_comm/inbox/`

Direction: Mac initiates all SCP transfers. The server cannot push to the Mac directly (SSH into the server is outbound from the Mac). Files placed in the Mac outbox are copied to the server inbox manually when a waldhomeserver task needs to be triggered.

This protocol is used for AI agent communication (dispatching Team 99 tasks via the server) but also for operational coordination: if the server-side pipeline encounters an issue that needs operator attention, the operator reads the report from the server outbox.

Monitoring and Alerting

SFA does not have external monitoring. The operator (Nimrod) monitors the system through:

- **Admin dashboard** – visible pipeline status, today's run summary, recent alerts
- **Log entries** in the database – each pipeline run writes structured logs accessible in the admin
- **Staleness indicator** – if the public page shows `warning` or `stale`, the publish pipeline has not run recently
- **manifest_last_good.json** – if the public page loads the last-good fallback rather than a fresh report, the recent publish either failed or was not uploaded

No automated email or push alerts in V1. Operator checks the admin dashboard regularly.

Backup and Recovery

Database backup:

- PostgreSQL dump via `pg_dump` – recommended daily, stored on the attached HDD
- Schema is fully reproducible via Alembic migrations from an empty database
- Catalog data (products, aliases, scope-skip rules) is in the DB – a backup is necessary to avoid rebuilding 232 aliases and 301 scope-skip rules from scratch

Published artifacts:

- All timestamped artifacts remain on the uPress server (`wp-content/uploads/market/`)
- Any past artifact can be referenced directly via its timestamp URL
- No TTL on uploaded artifacts – historical versions persist indefinitely

Recovery scenario:

- If waldhomeserver is unavailable, the WordPress frontend falls back to `manifest_last_good.json` and the last successfully uploaded artifact – the public index remains available, just not updated

- If uPress is unavailable, the public index is offline, but the local pipeline continues to run and produce artifacts; they are uploaded when hosting is restored